

Modul 4 *State Hoisting*, Perubahan Konfigurasi, dan *Side-effect*

1.1 Tujuan Pembelajaran

1. Memahami konsep state-hoisting dalam membangun struktur program aplikasi Android dengan Jetpack Compose yang reusable.
2. Memahami fungsi konstruk `rememberSaveable()` untuk menyimpan nilai state ketika terjadi perubahan konfigurasi
3. Memahami fitur side-effect dengan `LaunchedEffect` dan menerapkannya pada kasus-kasus tertentu yang hanya dapat dilakukan melalui side-effect pada aplikasi Android dengan Jetpack Compose.

1.2 Dasar Teori

1.2.1 State Hoisting

Tujuan dari *state hoisting* di Jetpack Compose adalah untuk menciptakan komponen UI yang lebih bersih, lebih mudah dipelihara, dan dapat digunakan kembali dengan memindahkan manajemen state dari komponen anak ke komponen induknya. Hal ini menetapkan sumber kebenaran tunggal, memastikan aliran data berjalan satu arah, dan meningkatkan pengujian dengan membuat elemen Composable menjadi *stateless*.

Tujuan dan keunggulan utama dari *state hoisting* meliputi:

- Kemudahan penggunaan kembali: Komponen *stateless composables* lebih mudah digunakan kembali di berbagai bagian aplikasi.
- Kemudahan pengujian: Karena status diteruskan dan peristiwa berupa fungsi callback, komponen UI lebih mudah untuk dilakukan unit test dan direview.
- Single Source of Truth (SSoT): Hoisting mencegah inkonsistensi data dengan mengelola status di satu lokasi pusat, memungkinkan induk untuk mengontrol atau mengamati perubahan.
- Menerapkan konsep Separation of Concern: Memisahkan logika UI (presentasi) dari logika bisnis (manajemen state), sehingga kode program menjadi lebih bersih.
- Alur Data Satu Arah: Status mengalir ke bawah (melalui parameter) dan event mengalir ke atas (melalui fitur callback/lambda).

Sebagai contoh, alih-alih `TextField` mengelola input teksnya sendiri, ia menerima string dan callback `onValueChange` dari induknya, sehingga `TextField` dapat digunakan kembali untuk berbagai jenis input. *State hoisting* di Jetpack Compose memindahkan state composable anak ke induknya, membuat composable anak menjadi *stateless*, sehingga dapat di-*reuse*, dan dapat diuji secara independen. State hoisting ini dilakukan dengan melibatkan penggantian *state* internal sebuah composable dengan parameter (misal nilai state composable tertentu) dan callback event (fungsi tertentu yang dijalankan atas event suatu composable, seperti

`onValueChange`). Dengan demikian menciptakan efek SSoT, di mana data mengalir ke bawah (dari composable induk ke anak) dan event mengalir ke atas (dari composable anak ke induk).

Sebagai contoh, kode program berikut ini sebelum dilakukan state-hoisting:

```
@Composable
fun Counter() {
    // State is local and trapped
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}
```

Setelah dilakukan state hoisting, kode programnya berubah menjadi:

```
@Composable
fun Counter(count: Int, onIncrement: () -> Unit) {
    Button(onClick = onIncrement) {
        Text("Count: $count")
    }
}
```

Dan kode program composable induknya menjadi seperti:

```
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(0) }
    // Passing state down, receiving events up
    Counter(count = count, onIncrement = { count++ })
}
```

1.2.2 Perubahan Konfigurasi

Meskipun `remember` membantu Anda mempertahankan nilai state di seluruh proses rekomposisi UI dengan Jetpack Compose, nilai *state* tersebut tidak dipertahankan

ketika terjadi perubahan konfigurasi. Untuk itu, Anda harus menggunakan

`rememberSaveable` dan `rememberSaveable` akan secara otomatis menyimpan nilai apa pun yang dapat disimpan dalam sebuah objek `Bundle`. Untuk nilai lain, Anda dapat memberikan objek saver secara custom.

API `rememberSaveable` berperilaku mirip dengan `remember` karena mempertahankan status di seluruh rekomposisi, dan juga di seluruh pembuatan

ulang aktivitas atau proses menggunakan mekanisme status instance yang tersimpan. Sebagai contoh, perubahan konfigurasi yang terjadi ketika *device*/layar diputar (dirotasi).

```
@Composable
fun NameInput() {
    // This name survives rotation!
    var name by rememberSaveable { mutableStateOf("") }

    TextField(
        value = name,
        onValueChange = { name = it },
        label = { Text("Enter your name") }
    )
}
```

Sebagai catatan: `rememberSaveable` tidak akan mempertahankan nilai *state* jika aplikasi/Activity ditutup sepenuhnya oleh pengguna. Sebagai contoh, *state* tidak akan dipertahankan jika pengguna menggeser Activity yang saat ini digunakan ke atas dari layar Activity yang terbaru.

1.2.3 Side-effect

LaunchedEffect adalah fungsi composable yang meluncurkan coroutine yang tercakup dalam siklus hidup composable. Fungsi ini secara otomatis membatalkan coroutine ketika composable meninggalkan komposisi atau ketika parameter kuncinya berubah. LaunchedEffect merupakan salah satu fitur dari API terkait side-effect yang terdapat di dalam pemrograman aplikasi Android yang menggunakan Jetpack Compose. Side-effect dalam konteks ini mengacu pada operasi-operasi yang:

- Mempengaruhi sesuatu di luar fungsi composable
- Perlu dipicu/dijalankan berdasarkan perubahan *state* tertentu
- Prosesnya perlu tercakup dalam siklus hidup Composable tertentu.

Misalnya, jika Anda perlu mengambil data dari Internet secara otomatis ketika tampilan pada layar muncul, memulai animasi (seperti *fading*, *scrolling*, *sliding* terhadap suatu elemen Composable) ketika suatu nilai variabel berubah, atau mendengarkan aliran data yang datang, maka penggunaan LaunchedEffect adalah alat yang tepat untuk melakukan hal-hal tersebut dalam lingkup Composable (UI). Dapat dibayangkan seperti ini: jika Anda perlu menjalankan kode secara asinkron yang harus terikat pada siklus hidup Composable, maka penggunaan LaunchedEffect dapat memastikan hal ini terjadi dengan bersih tanpa perlu khawatir akan terjadinya kebocoran memori (*memory leaks*).

Berikut ini adalah contoh penggunaan `LaunchedEffect` untuk mengambil data tertentu pada saat pertama kali program berjalan:

```
import androidx.compose.runtime.*
import kotlinx.coroutines.delay

@Composable
fun MyDataScreen(userId: String) {
    var data by remember { mutableStateOf("Loading...") }

    // This effect runs only when MyDataScreen first enters the
    composition
    LaunchedEffect(Unit) {
        // Simulate a network request
        delay(2000)
        data = "Data Loaded for user $userId"
    }

    Text(text = data)
}
```

Dari contoh kode program tersebut, pertama kali composable `Text` akan menampilkan pesan "Loading...", setelah dua detik, teks yang ditampilkan akan berubah menjadi "Data loaded for user \$userId". Proses pengambilan data dilakukan secara otomatis pada saat aplikasi pertama kali dijalankan.

1.3 Rangkuman

State hoisting adalah pola desain di mana kita memindahkan state (data yang bisa berubah) dari dalam sebuah fungsi `Composable` ke pemanggilnya (caller). Ini mengubah `Composable` dari "Stateful" (mengelola datanya sendiri) menjadi "Stateless" (menerima data melalui parameter).

Mengapa penting? Membuat `Composable` lebih mudah dites, bisa digunakan kembali (reusable), dan menjadi satu-satunya sumber kebenaran (single source of truth).

Dalam cara kerjanya, biasanya melibatkan dua parameter:

- `value: T`: Data yang akan ditampilkan.
- `onValueChange: (T) -> Unit`: Event untuk meminta perubahan data.

`LaunchedEffect` adalah fungsi Side Effect yang memungkinkan kita menjalankan kode (biasanya yang bersifat asynchronous) di dalam cakupan `Composable` secara aman. Cara kerjanya ia akan menjalankan blok kode program saat pertama kali

masuk ke komposisi. Jika "key" yang diberikan berubah, ia akan membatalkan proses yang lama dan menjalankannya kembali. Kapan digunakan? Ketika memanggil API saat layar dibuka, menampilkan Snackbar, melakukan navigasi antar layar berdasarkan perubahan state. `LaunchedEffect` akan secara otomatis dibatalkan jika `Composable` keluar dari komposisi (`onCleared`), sehingga mencegah kebocoran memori.

Sebagaimana diketahui, `remember` menjaga state tetap ada saat terjadi recomposition. Namun, `remember` akan hilang jika aktivitas hancur (misalnya saat rotasi layar). `rememberSaveable` hadir untuk mengatasi hal tersebut. `rememberSaveable` menyimpan data ke dalam Bundle. Data akan tetap aman meskipun terjadi rotasi layar atau jika sistem mematikan aplikasi untuk menghemat RAM. Kelebihannya, sangat sesuai untuk data ringan seperti input teks di Form, status checkbox, atau posisi scroll. Namun, keterbatasannya hanya bisa menyimpan tipe data yang bisa dimasukkan ke Bundle secara otomatis. Untuk objek kompleks, diperlukan penggunaan Saver.

1.4 Alat dan Bahan Praktik

1.4.1 Perangkat keras

- Komputer/Laptop dengan RAM minimal 8 GB, 16 GB direkomendasikan.
- Processor Dual Core.
- Perangkat Android fisik, untuk menjalankan program jika RAM komputer yang digunakan tidak memadai untuk menjalankan Android Emulator di dalam Android Studio.

1.4.2 Perangkat Lunak

- Android Studio dengan Android SDK dan Android Emulator yang telah terinstall.
- Materi Pembelajaran Kelas.
- Modul Praktikum.

1.5 Percobaan

1.5.1 State Hoisting

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template **Empty Activity** untuk **Phone and Tablet**. Pastikan project yang dibuat menggunakan Jetpack

Compose. Tunggu sampai proses Build inialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.

2. Buka file `MainActivity.kt` dan tambahkan fungsi composable berikut ini di **luar** class `MainActivity`:

```
fun CounterScreen(modifier: Modifier) {
    var number by remember { mutableStateOf(0) }
    val label by remember { mutableStateOf("Increment") }
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = { number++ }) {
            Text(text = "$label")
        }
    }
}
```

Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.runtime.remember
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.material3.Button
import androidx.compose.foundation.layout.Column
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

3. Buka file `MainActivity.kt`, temukan fungsi `onCreate()` yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah bagian kode program berikut di dalam fungsi `onCreate()`:

```
Greeting(
    name = "Android",
    modifier = Modifier.padding(innerPadding)
)
```

Kemudian ubah bagian tersebut menjadi:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

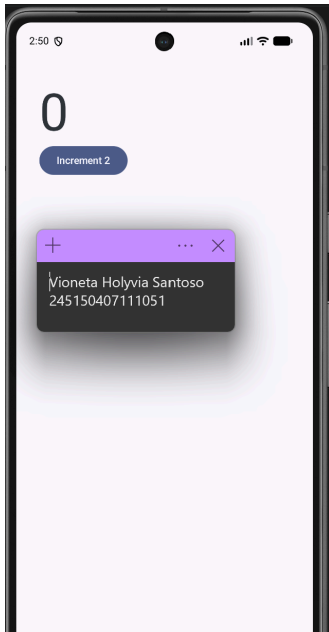
Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-*compile*.

Aktivitas Percobaan

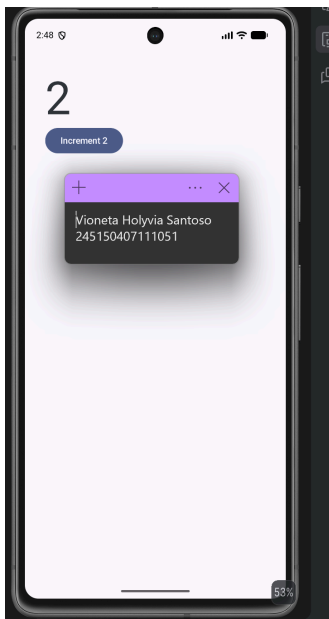
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.
 - a. Apa yang terjadi setiap kali tombol "Increment" diklik?
Nilai angka pada perangkat Android fisik bertambah 1 secara berurutan. Yang awalnya 0 menjadi 1, 1 menjadi 2, dan seterusnya.
 - b. Apa yang menyebabkan hal tersebut terjadi?
Hal tersebut terjadi karena Event `onClick` pada `Button` mengeksekusi lambda `{ number++ }`. Perubahan nilai pada `mutableStateOf` ini memicu **rekomposisi** (*recomposition*) pada Composable `CounterScreen`, sehingga UI diperbarui dengan nilai `number` terbaru.
 - c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
Variabel `number` (`var number by remember { mutableStateOf(0) }`) yang berperan menampilkan perubahan angka pada Composable Text.
 - d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
Variabel `label` (`val label by remember { mutableStateOf("Increment") }`) yang berperan untuk menampilkan label teks pada Composable Button
2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
 - a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?
Perubahan yang perlu dilakukan adalah:
 - Mengubah Label Tombol : Mengubah inisialisasi state `label` dari `"Increment"` menjadi `"Increment 2"`.
 - Mengubah Logika Penambahan (Increment): Mengubah logika pada event `onClick` tombol dari `number++` (bertambah 1) menjadi

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Before



After



3. Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable [CounterScreen](#).

Ubah signature kode program pada bagian fungsi Composable `CounterScreen` dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

menjadi:

```
fun CounterScreen(modifier: Modifier,  
                  number: Int, label: String, onClick: () ->  
                  Unit)
```

Ubah *body* di dalam Composable `CounterScreen` menjadi:

```
// var number by remember { mutableStateOf(0) }  
// val label by remember { mutableStateOf("Increment") }  
Column(modifier) {  
    Text(text = "$number", fontSize = 72.sp)  
    Button(onClick = onClick) {  
        Text(text = "$label")  
    }  
}
```

Ubah pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

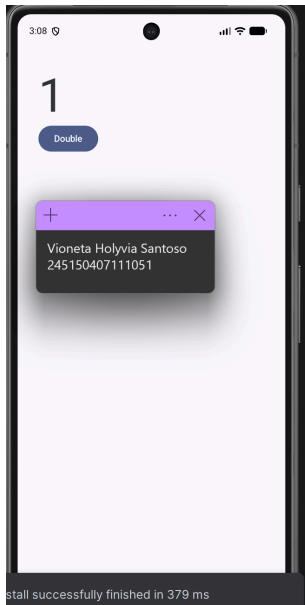
menjadi:

```
var number by remember { mutableStateOf(0) }  
CounterScreen(Modifier.padding(32.dp),  
    number = number,  
    label = "Increment",  
    onClick = { number++ }  
)
```

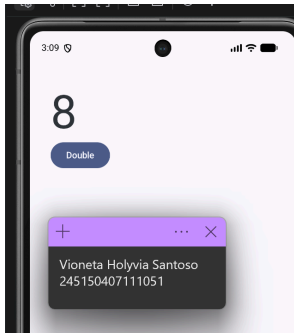
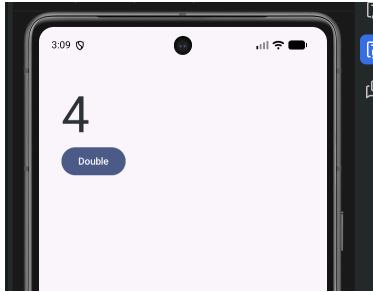
Perhatikan bahwa deklarasi variabel state `number` dari yang sebelumnya dilakukan di dalam fungsi Composable `CounterScreen`, sekarang dideklarasikan di dalam fungsi `onCreate()` sebelum fungsi Composable `CounterScreen` dipanggil dari dalam fungsi `onCreate()`.

4. Jalankan program, dan kemudian klik beberapa kali tombol `Increment` sehingga angka yang ditampilkan berubah.

- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
Iya, program masi sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya
 - b. Di bagian kode program manakah saat ini proses increment dilakukan?
Proses increment saat ini dilakukan di dalam fungsi `onCreate()`, tepatnya di Composable `CounterScreen`: yang terletak pada kode `onButtonClick = { number++ }`
 - c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?
 - State composable text dideklarasikan di dalam fungsi `onCreate()` sebelum pemanggilan `CounterScreen`
 - State Composable Button "Increment" dideklarasikan langsung sebagai nilai string literal `"Increment"` pada argumen parameter `label` saat pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()`
5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:
- a. Label tombol "Increment" menjadi "Double"
 - b. Nilai awal variabel `number` dari `0` menjadi `1`.
 - c. Ubah fungsi/lambda `onButtonClick` pada pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
 - d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.
6. Jalankan program,
- a. Perhatikan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.
Sebelum digandakan



Setelah digandakan



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Seluruh perubahan pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan pada fungsi **onCreate()** (sebagai *parent* / pemanggil Composable).

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada perubahan di dalam fungsi Composable CounterScreen

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Hal ini membuktikan manfaat utama *state hoisting*, yaitu membuat komponen UI (CounterScreen) bersifat stateless, decoupled, dan reusable (dapat digunakan kembali). Ketika terjadi perubahan logika bisnis atau kebutuhan UI (seperti mengubah *increment* menjadi penggandaan angka atau mengganti label tombol), kita hanya perlu mengubah logika pada komponen induk (*caller*) tanpa harus memodifikasi atau merusak struktur kode komponen UI anak (CounterScreen). Komponen UI cukup fokus menerima data dan menampilkan UI (*presentation logic*), sedangkan pengelolaan data ditangani sepenuhnya oleh induknya (*Single Source of Truth*)

Tugas Praktik

Lembar Kerja Praktik

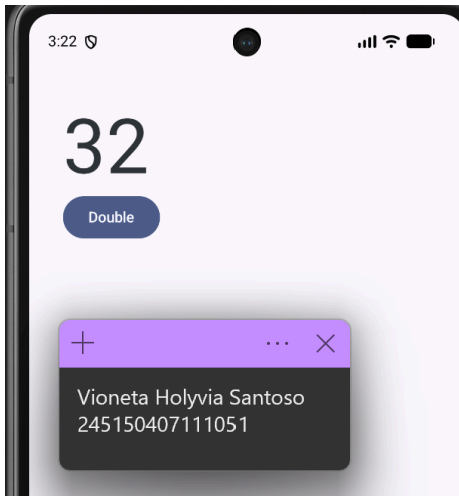
1.5.2 Perubahan Konfigurasi

Pra-Percobaan

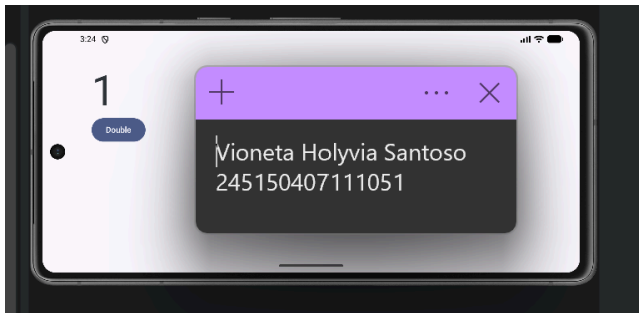
1. Untuk percobaan terkait Perubahan Konfigurasi aplikasi Android, gunakan Project Android Studio yang sama dengan kode program percobaan *State Hoisting* setelah menyelesaikan tahap nomor 6 percobaan tersebut.
2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel number).

Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.
Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
 Angka 1, Hal ini terjadi karena ketika posisi perangkat diputar dari *portrait* ke *landscape*, Android melakukan perubahan konfigurasi (*configuration change*) yang memicu pembuatan ulang Activity (*Activity re-creation*). Konstruksi `remember` hanya mempertahankan nilai *state* selama proses recomposisi UI, tetapi nilainya akan terhapus dan di-reset kembali ke nilai awal ketika Activity dihancurkan dan dibuat ulang.

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

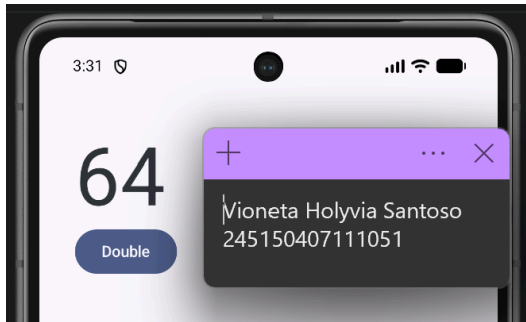
```
var number by remember { mutableStateOf(1) }
```

menjadi:

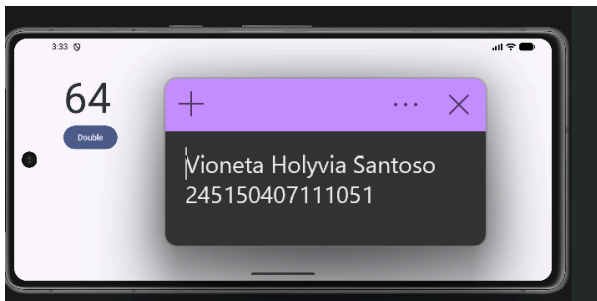
```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela *editor* Android Studio).

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi *landscape*.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
64, dimana sama dengan versi *portrait*. Hal ini terjadi karena penggunaan konstruk `rememberSaveable` yang membuat nilai state `number` disimpan ke dalam objek `Bundle` melalui mekanisme *saved instance state*. Ketika terjadi perubahan konfigurasi (seperti rotasi layar dari *portrait* ke *landscape*) yang mengakibatkan Activity dibuat ulang (*re-created*), sistem Android memulihkan (*restore*) nilai state tersebut dari `Bundle`, sehingga nilainya tidak ter-reset ke nilai awal

- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?
- `remember`: Hanya menyimpan/mempertahankan nilai state selama proses rekomposisi (*recomposition*) UI. Nilai state akan hilang/ter-reset jika Activity dihancurkan akibat perubahan konfigurasi (seperti rotasi layar atau perubahan tema gelap/terang).
 - `rememberSaveable`: Mempertahankan nilai state saat proses rekomposisi DAN saat terjadi perubahan konfigurasi (*configuration change*). Data disimpan secara otomatis ke dalam `Bundle` sehingga tetap aman meskipun Activity dibuat ulang oleh sistem.
 -
4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!
- Contoh penerapannya
 1. Input Form / Text Field: Menyimpan teks yang sedang diketik pengguna pada form registrasi/login (misalnya input Email, Nama, atau Alamat) agar teks tidak terhapus saat pengguna memutar posisi layar HP
 2. Elemen UI Toggle / Checkbox: Mempertahankan status *checked* / *unchecked* pada checkbox persetujuan syarat & ketentuan atau toggle switch opsi pengaturan.
 3. Navigasi Tab / Pilihan Filter: Menyimpan indeks Tab yang sedang aktif dipilih atau status filter pencarian produk yang sedang diterapkan pengguna.
 - Manfaat yang Diperoleh:
 1. Meningkatkan Pengalaman Pengguna (User Experience / UX): Pengguna tidak merasa frustrasi karena harus mengulang mengetik atau mengisikan kembali data input dari awal saat tidak sengaja memutar posisi layar.
 2. Integritas Data Sementara: Memastikan data interaksi sementara pada antarmuka pengguna (*UI State*) tetap konsisten sepanjang siklus hidup tampilan aplikasi

Tugas Praktik

Lembar Kerja Praktik

1.5.3 LaunchedEffect

Pra-Percobaan 1

1. Untuk percobaan ini, gunakan Project Android Studio yang sama dengan kode program percobaan Perubahan Konfigurasi setelah menyelesaikan tahap nomor 4 percobaan tersebut.
3. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel `number`).
4. Mengaitkan composable `SnackBar` di dalam `Scaffold`.
Di dalam fungsi `onCreate()` dan di dalam lambda `Theme{}` sebelum pemanggilan composable `Scaffold` dilakukan, tambahkan deklarasi host `state` untuk menyimpan state composable `SnackBar` yang akan digunakan untuk menampilkan pesan:

```
val snackbarHostState = remember { SnackbarHostState() }
```

Kemudian ubah bagian pemanggilan fungsi Composable `Scaffold` di dalam fungsi `onCreate()` yang sebelumnya:

```
Scaffold(modifier = Modifier.fillMaxSize())
```

menjadi:

```
Scaffold(  
    modifier = Modifier.fillMaxSize(),  
    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }  
)
```

5. Menerapkan fitur Side-effect dengan `LaunchedEffect`. Dengan posisi cursor tetap berada di dalam lambda `Scaffold`, setelah pemanggilan composable `CounterScreen` tambahkan kode program *side-effect* berikut ini:

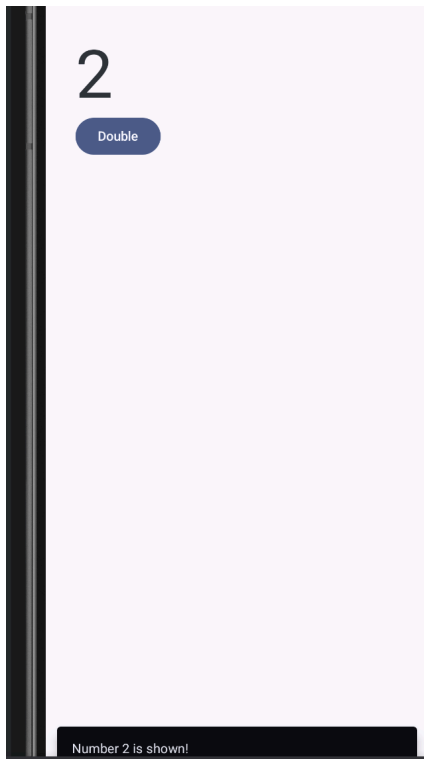
```
LaunchedEffect(number) {  
    snackbarHostState.showSnackBar("Number $number is shown!")  
}
```

6. Lakukan *import* semua *dependency* yang diperlukan dari perubahan kode program yang telah dilakukan (dengan melakukan *refactoring* melalui Alt-Enter pada bagian yang ditandai merah di dalam jendela editor Android Studio) agar perubahan kode program yang dilakukan dapat di-*compile* dan dijalankan dengan baik.

Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composable CounterScreen().

Aktivitas Percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?



- Layar menampilkan angka 1, tombol bertuliskan "Double", serta sebuah Snackbar di bagian bawah bertuliskan "Number 1 is shown!"
- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Pesan tersebut ditampilkan karena `LaunchedEffect` selalu langsung dijalankan setidaknya satu kali saat Composable pertama kali memasuki komposisi (*initial composition*).
 - c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Hal yang memicu pesan pada snackbar tersebut ditampilkan adalah inisialisasi nilai awal dari variabel `number` (yaitu 1) memicu `LaunchedEffect(number)` untuk dieksekusi. Bagian kode programnya adalah

```

LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}

```

2. Klik button "Double" satu kali,

- a. Apa yang terjadi?

Ketika button "Double" diklik satu kali, nilai angka di layar berubah dari **1** menjadi **2**, lalu SnackBar lama digantikan oleh SnackBar baru bertuliskan "Number 2 is shown!".

- b. Mengapa hal tersebut terjadi? Jelaskan!

Hal tersebut terjadi ketika tombol "Double" diklik, fungsi `onButtonClick` mengeksekusi `number *= 2`. Perubahan nilai pada `state number` mendeteksi mutasi pada parameter `key` di `LaunchedEffect(number)`, yang memicu pembatalan `coroutine` lama dan menjalankan kembali blok kode SnackBar dengan nilai `number` yang baru.

Pra-Percobaan 2

3. Untuk percobaan ini, lanjutkan Project Android Studio yang sama dengan kode program percobaan Side-effect sebelumnya.
4. Di dalam package yang sama dengan class `MainActivity`, buat sebuah class Kotlin baru bernama `Repo` (file: `Repo.kt`) dengan struktur class dan fungsi static `getData()` seperti pada kode program berikut ini:

```

class Repo {
    companion object {
        suspend fun getData() : Int {
            delay(2000) // simulating network
            return Random.nextInt(100, 1000)
        }
    }
}

```

Lakukan import dependency untuk fungsi `delay()` dan class `Random`. Fungsi `delay(2000)` ini digunakan untuk menyimulasikan waktu yang dibutuhkan untuk mengambil data melalui koneksi internet.

5. Di dalam fungsi `onCreate()`, setelah `LaunchedEffect` pada percobaan sebelumnya, tambahkan deklarasi fungsi `LaunchedEffect` baru berikut ini:

```

LaunchedEffect(Unit) {
    number = Repo.getData()
}

```

6. Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composible CounterScreen().

Aktivitas Percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Layar pertama kali menampilkan angka **1**, tombol bertuliskan "Double", serta sebuah *SnackBar* di bagian bawah yang bertuliskan "Number 1 is shown!"
 - b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah 2 detik, angka di layar berubah secara otomatis menjadi angka acak antara 100 hingga 999. Disini saya mendapatkan angka **584** & pesan *SnackBar* berganti menjadi pesan baru bertuliskan "Number 584 is shown!"
 - c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada *SnackBar* tersebut ditampilkan? Pada bagian kode program yang mana?
 - Perubahan Angka
 - Pemicu Perubahan Angka: Dipicu oleh `LaunchedEffect(Unit)` yang mengeksekusi fungsi `suspend Repo.getData()`. Fungsi ini menunda proses selama 2 detik (`delay(2000)`), lalu menghasilkan angka acak yang kemudian mengubah nilai variabel `state number`.
 - Bagian Kode

```
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```
 - Pesan *SnackBar* Baru
 - Pemicu Pesan *SnackBar* Baru: Perubahan nilai variabel `number` mendeteksi adanya mutasi *key* pada `LaunchedEffect(number)`, sehingga memicu `LaunchedEffect` tersebut untuk menjalankan ulang perintah pembacaan *SnackBar* dengan nilai `number` terbaru.
 - Bagian Kode:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackBar("Number
$number is shown!")
}
```

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,
- Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?
 - `LaunchedEffect(state)`: Berjalan saat pertama kali `Composable` masuk ke komposisi DAN akan dibatalkan lalu dijalankan ulang setiap kali nilai variabel `state` (*key*) yang dipantau mengalami perubahan.
 - `LaunchedEffect(Unit)`: Berjalan hanya satu kali saja saat `Composable` pertama kali masuk ke komposisi (*initial composition*). Karena `Unit` adalah nilai konstan yang tidak pernah berubah, blok di dalamnya tidak akan pernah dieksekusi ulang ketika terjadi rekomposisi
 - Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?
 - `LaunchedEffect(state)`: Dijalankan saat inisialisasi awal serta setiap kali terjadi mutasi/perubahan nilai pada variabel `state`. Sangat cocok digunakan untuk merespons perubahan data dinamis (seperti memperbarui UI, menampilkan alert/SnackBar saat data berubah, atau memicu animasi berdasarkan perubahan `state`).
 - `LaunchedEffect(Unit)`: Dijalankan tepat satu kali saat layar/komponen pertama kali dibuka/dimuat. Sangat cocok untuk tugas *one-time event* / inisialisasi awal (seperti mengambil data dari API/Internet, meminta izin akses, atau membaca data dari basis data lokal saat layar dibuka).

1.6 Referensi

Google (2024) *The Activity Lifecycle*. Available at:

<https://developer.android.com/guide/components/activities/activity-lifecycle>

Google (2024) *Jetpack Compose layouts*. Available at:

<https://developer.android.com/jetpack/compose/layouts>

Google (2024) *Compose Modifiers*. Available at:

<https://developer.android.com/jetpack/compose/modifiers>

1.7 Glosarium

Activity Lifecycle

Rangkaian tahapan siklus hidup aplikasi dari inisialisasi (onCreate), berjalan (onResume), hingga dimatikan (onDestroy).

Intent

Objek pesan yang digunakan untuk meminta tindakan dari komponen aplikasi lain (misal membuka tautan WhatsApp).

Jetpack Compose

UI Toolkit deklaratif modern berbasis Kotlin untuk membangun antarmuka Android.

@Preview

Anotasi Compose yang memungkinkan pengembang mempratinjau UI langsung di Android Studio tanpa emulator.

Modifier

Kelas untuk mengatur dekorasi, ukuran, tata letak, dan perilaku sebuah elemen Compose.

Modifier Chaining

Merangkai fungsi Modifier secara berurutan, di mana urutannya sangat menentukan hasil akhir (order matters).

clip()

Fungsi Modifier yang digunakan untuk memotong batasan elemen sesuai dengan bentuk yang ditentukan (misal: CircleShape).

Column

Layout penampung untuk menyusun elemen anak secara vertikal.

Row

Layout penampung untuk menyusun elemen anak secara horizontal.

Box

Layout penampung untuk menumpuk elemen anak berdasarkan layer/kedalaman (z-index).

